❼ Ensures that this Tag Helper runs before any others attached to the element

Instead of a standalone `<if>` element, the Razor engine executes the `IfTagHelper` whenever it finds an element with an `if` attribute. This can be applied to any HTML element: `<p>`, `<div>`, `<input>`, whatever you need. You should define a Boolean property specifying whether you should render the content, which is bound to the value in the `if` attribute.

The `Process()` function is much simpler here. If `RenderContent` is `false`, it sets the `TagHelperOutput.TagName` to `null`, which removes the element from the page. It also calls `SuppressOutput()`, which prevents any content inside the attributed element from being rendered. If `RenderContent` is `true`, you skip these steps, and the content is rendered as normal.

One other point of note is the overridden `Order` property. This controls the order in which Tag Helpers run when multiple Tag Helpers are applied to an element. By setting `Order` to `int.MinValue`, you ensure that `IfTagHelper` always runs first, removing the element if required, before other Tag Helpers execute. There's generally no point running other Tag Helpers if the element is going to be removed from the page anyway.

NOTE Remember to register your custom Tag Helpers in _ViewImports .cshtml with the `@addTagHelper` directive.

With a simple HTML attribute, you can now conditionally render elements in Razor templates without having to fall back to C#. This Tag Helper can show and hide content without needing to know what the content is. In the next section we'll create a Tag Helper that *does* need to know the content.